

Assessment 1 - Solutions

Annexure A: Analytical Problem Solving

Question 1: Water Jug Problem

State notation: (x, y) where x = water in the 4-gallon jug, y = water in the 3-gallon jug.

Step	Action	State (4-gal, 3-gal)
0	Start	(0, 0)
1	Fill the 4-gallon jug	(4, 0)
2	Pour 4-gallon jug into 3-gallon jug	(1, 3)
3	Empty the 3-gallon jug	(1, 0)
4	Pour 4-gallon jug into 3-gallon jug	(0, 1)
5	Fill the 4-gallon jug	(4, 1)
6	Pour 4-gallon jug into 3-gallon jug	(2, 3)

Result: After step 6, the 4-gallon jug contains exactly 2 gallons of water.

Question 2: Mars Rover Agent

(i) Percepts

Camera images of the terrain, spectrometer/chemical composition readings, GPS/position coordinates, terrain slope and obstacle-detection sensor data, temperature and atmospheric pressure readings, wheel odometry/inertial measurements, battery/power level, and command signals received from Earth.

(ii) Operating Environment

Partially observable - sensors only cover a limited local range; **Stochastic** - dust storms and terrain hazards are unpredictable; **Sequential** - current actions affect future states (e.g., battery use, position); **Dynamic** - weather and lighting change while the rover deliberates; **Continuous** - motion, sensor readings, and time are continuous; **Unknown** - the terrain map is not fully known in advance.

(iii) Actions

Move/steer in a direction, extend or retract the robotic arm, drill into rock/soil, collect a sample, run onboard chemical analysis, capture an image, transmit data to an orbiter or Earth, recharge via solar panels, and perform obstacle avoidance manoeuvres.

(iv) Performance Evaluation

Distance safely traversed, number and scientific value of samples analysed, accuracy/quality of data returned, energy efficiency (power consumed per task), avoidance of damage or getting stuck, task-completion time,

and successful data transmission rate back to Earth.

(v) Most Suitable Agent Architecture

A **model-based, utility-based agent** is most suitable. Since the environment is partially observable and dynamic, the rover must maintain an internal model (a map and history of visited terrain) built from its percepts to make sound decisions despite limited direct observation. Because it must choose among multiple competing objectives - e.g., which sample to prioritise under limited battery and time - a utility function that ranks actions by expected value is more appropriate than a simple goal-based agent, which can only judge whether a goal is reached or not.

Question 3: 8-Queens Problem

Problem Formulation

- **State:** Any arrangement of 0 to 8 queens on the board. Using the complete-state formulation, every state has exactly one queen per column.
- **Initial state:** A full configuration with one queen placed in each column (e.g., all in row 1), which may initially contain attacking pairs.
- **Actions:** Move a queen already on the board to a different row within the same column (only one queen moves per action).
- **Goal test:** No two queens attack each other - i.e., no two queens share the same row, column, or diagonal.
- **Path cost:** Not relevant here; the 8-Queens problem is treated as a constraint-satisfaction / local-search problem rather than a shortest-path search problem.

Search Strategy

Because the given configuration starts as a complete-but-invalid arrangement (the queen in column 1 attacks the queen in column 8 diagonally), the natural approach is **local search**, specifically **hill-climbing with the min-conflicts heuristic**: at each step, select a queen involved in a conflict and move it to the row within its column that minimises the total number of attacking pairs. Repeat until zero conflicts remain, or restart if stuck in a local minimum. Alternatively, a systematic **backtracking search** (placing one queen per column and pruning invalid partial states) is guaranteed to find a solution and is what the accompanying Python program uses.

One Valid Solution (found via backtracking search)

```
Q . . . . . . .
. . . . Q . . .
. . . . . . Q
. . . . . Q .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .
```

Column position of the queen in each row (0-indexed): [0, 4, 7, 5, 2, 6, 1, 3]

Question 4: OLA Cab Booking Agent

Since the customer chooses among several satisfactory options (mini, micro, sedan, shared, prime) based on trade-offs between fare, wait time, and comfort - rather than a single fixed goal - a **utility-based agent** is the correct choice. A plain goal-based agent only distinguishes whether a goal is met, with no notion of preference between multiple valid options.

Pseudocode

```
function OLA_CAB_AGENT(percept) returns action
persist: state, cab_types, utility_function

state <- UPDATE_STATE(state, percept)
pickup <- state.pickup_location
destination <- state.destination
available_cabs <- GET_AVAILABLE_CABS(pickup, cab_types)

best_cab <- null
best_utility <- -infinity

for each cab in available_cabs do
utility <- UTILITY_FUNCTION(
fare = cab.price,
comfort = cab.comfort_level,
wait_time = cab.eta,
capacity = cab.seats,
preference = state.customer_preference)
if utility > best_utility then
best_utility <- utility
best_cab <- cab

action <- BOOK_CAB(best_cab, pickup, destination)
return action
```

Question 5: Uniform Cost Search (UCS)

Note: The assignment figure's delivery-network graph was only partially visible in the source scan. The graph below is used to demonstrate the full UCS method end-to-end; substitute your actual edge weights if they differ, and re-run Python_Code.py to regenerate this result.

Graph used (edges and weights)

- S -> A (2)
- S -> C (4)
- A -> B (3)
- A -> D (6)
- B -> D (3)
- B -> G (7)
- C -> D (2)
- D -> G (2)

UCS Expansion Trace

Node Expanded	Cumulative Cost from S
S	0
A	2
C	4
B	5
D	6
G	8

UCS always expands the frontier node with the lowest cumulative path cost (using a priority queue / min-heap). It expands S(0), then A(2) and C(4) become frontier options, A(2) expands next revealing B(5) and D(8-via-A), then C(4) expands giving a cheaper route to D(6), then B(5) expands, then D(6) expands revealing G(8), and finally G is popped as the goal.

Least-cost path: S -> C -> D -> G, with total cost = 8.

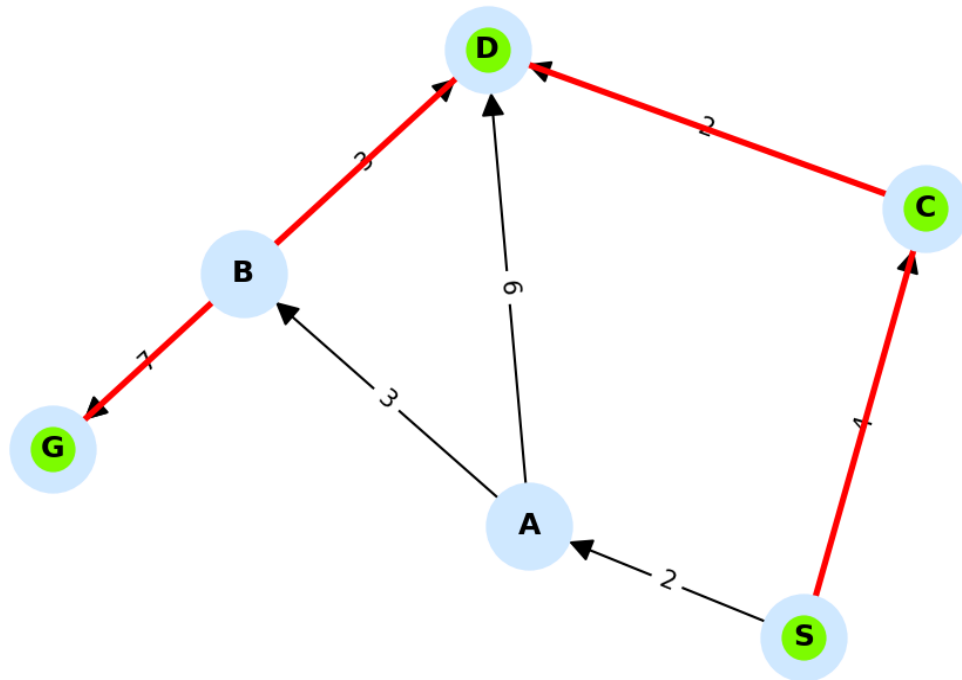


Figure: Delivery network graph with the least-cost path highlighted in red/green (generated by Python_Code.py, saved as Output.png).